Messaging - Week 1 class diagram

Solid boxes exist on the branch. Dashed boxes are Week 2 proposals; no live messaging service is supplied yet.

renders via

uses

future implementation uses

returns

controller.messaging.MessageController

- service: MessageService
- view: MessageView
+ constructor(service, view)
+ listAgents(): void
+ sendMessage(agentId, body): void
+ showInbox(): void
Tested delegation scaffold.
No username can be supplied by the UI.

model.messaging.MessageService [interface]

+ getAgents(): List<Agent>
+ sendMessage(agentId, body): Message
+ getInbox(): List<Message>
Contract: bound to one verified session.
No admin override for inbox reads.
Reject ambiguous case-insensitive identity.

view.messaging.MessageView [interface]

+ showAgents(List<Agent>)
+ showInbox(List<Message>)
+ showMessageSent(Message)
+ showError(String)
Terminal view contract only.

model.messaging.Message

- id: long (0 = unsaved)
- senderUsername: String
- recipientAgentId: int
- recipientName: String (snapshot)
- body: String
- sentAt: LocalDateTime
+ constructor(all fields)
+ getter for every field
Immutable; no setters.

model.messaging.MessageStore [interface]

+ save(Message): Message
+ findForRecipient(name): List<Message>
Generated IDs, ascending inbox order.
Case-insensitive recipient query.
Low-level store is not authorization.

Week 2 implementations [planned]

SqliteMessageStore
AuthenticatedMessageService
TerminalMessageView
Apply validated DDL at startup.
Wire authenticated terminal session.

Tests and integration status

MessageTest + MessageControllerTest execute now. Schema tests validate the proposed DDL. Abstract acceptance contract awaits a real Week 2 fixture.